

# CRSF: Collusion-Resilient Privacy-Preserving Sensor Fusion with Byzantine-Robust Participation

Chao Yin<sup>1,2</sup>, Haihong Tian<sup>3</sup>, Zheng Yang<sup>3</sup>, Haibin Zhang<sup>4,5</sup>, Fabio Massacci<sup>1,6</sup>, and Chenglu Jin<sup>2</sup>

<sup>1</sup> Vrije Universiteit Amsterdam, Amsterdam, the Netherlands  
c.yin@vu.nl, fabio.massacci@ieee.org

<sup>2</sup> Centrum Wiskunde & Informatica, Amsterdam, the Netherlands  
chenglu.jin@cw.nl

<sup>3</sup> Southwest University, Chongqing, China  
tianhaihong@email.swu.edu.cn, youngzheng@swu.edu.cn

<sup>4</sup> Yangtze Delta Region Institute of Tsinghua University, Zhejiang, China  
bchainzhang@aliyun.com

<sup>5</sup> Jiaxing Key Laboratory of Artificial Intelligence and Cyber Resilience, Zhejiang, China

<sup>6</sup> University of Trento, Trento, Italy

**Abstract.** Privacy-preserving sensor fusion enables an untrusted server to compute an aggregate result over distributed sensor measurements without learning either individual inputs or the final output. Recent garbled-circuit-based protocols provide an efficient realization of this functionality in a sensor-server-client architecture, but remain vulnerable to sensor-server collusion and Byzantine manipulation of sensor participation. These weaknesses can compromise honest-sensor privacy, incorrectly exclude honest sensors, and corrupt the computed fusion result, thereby undermining the security guarantees expected from the protocol. We present CRSF, a collusion-resilient sensor-fusion protocol that addresses these weaknesses while providing privacy, correctness with explicit abort, and liveness. CRSF introduces a Practical Byzantine Fault Tolerance (PBFT)-based agreement phase for sensor submissions and uses server-specific, status-dependent label release with threshold protection of circuit-input labels. This design prevents any Byzantine server from unilaterally manipulating sensor participation and prevents any admissible sensor-server coalition from obtaining enough secret material to compromise honest-sensor privacy.

We implement CRSF and compare its online execution time with the most relevant state-of-the-art baseline. Our Google Cloud evaluation measures the total computation and communication cost of the online protocol under fault-free and representative faulty executions. Across a range of fault-tolerant fusion circuits and up to 261 sensors, CRSF demonstrates a highly practical trade-off between robust security and protocol performance.

**Keywords:** Sensor Fusion · Secure Computation · Collusion Resilient.

## 1 Introduction

Sensor fusion is a fundamental task in cyber-physical systems, distributed sensing, and safety-critical monitoring applications. By combining readings from multiple sensors, sensor fusion provides a view of the physical environment that is more accurate, robust, and actionable than any individual measurement alone. Many sensor-fusion deployments can be abstracted as a three-role architecture, as seen in mobile crowdsensing, participatory sensing, and cloud-assisted sensing systems [13, 27, 16, 7, 5, 6]. Distributed sensors or mobile devices contribute measurements, a client or application specifies the fusion task and consumes the result, and an intermediate service platform collects, processes, or stores the data. This architecture imposes stringent security requirements: sensor readings may reveal sensitive information and should remain private from both the server and the client; corrupted sensors should not be able to arbitrarily bias or disrupt the computation; and, under the intended availability assumptions, the client should reliably obtain the fusion result.

Prior work on three-role sensing and aggregation systems has studied related security requirements from different angles, including privacy-preserving aggregation over sensor or mobile-device inputs, verifiable aggregation in the presence of corrupted aggregators or sensors, and robustness against false data or pollution attacks [22, 12, 4, 24, 19, 30]. In the more specific setting of privacy-preserving sensor fusion, PG [13] is a garbled-circuit-based protocol that combines input privacy with robustness against pollution attacks, and provides guaranteed output delivery under a bounded Byzantine model. To the best of our knowledge, PG is the first construction to simultaneously achieve input privacy, Byzantine fault tolerance, and guaranteed output delivery in a three-role sensor-fusion architecture. PG is therefore a particularly important baseline in this direction.

However, the security guarantees of PG do not extend to sensor-server collusion, where corrupted sensors and a Byzantine server combine their protocol views. This represents a significant gap, as collusion between data providers and aggregation-side entities is a well-documented threat in sensing systems [26, 11, 29, 20]. Such collusion is highly realistic in scenarios where a single adversary simultaneously compromises both a subset of sensors and a central server. This gap motivates us to revisit privacy-preserving sensor fusion under a stronger adversarial model, which accounts for both sensor-server collusion and Byzantine server behavior, while preserving the core properties of the existing baseline.

We identify two limitations of PG under this model. First, once sensor-server collusion is allowed, a corrupted sensor and a Byzantine server may combine their views to obtain complementary circuit-input labels on a circuit-input wire, which can compromise honest-sensor privacy and fusion result integrity. Second, PG relies on a single server to determine which sensor submissions will be used in data fusion. A Byzantine server can therefore manipulate the effective participating set of the sensors, for example by excluding honest sensors while retaining malicious ones, and thereby invalidate the fault-threshold premise required by the underlying fault-tolerant algorithm. These observations reveal two design requirements for strengthening garbled-circuit-based sensor fusion: participation

decisions must be correctly made, and circuit-input labels must be protected against sensor-server coalitions.

To meet these requirements, we propose CRSF, a collusion-resilient sensor-fusion protocol that follows the garbled-circuit workflow, like PG [13], while strengthening it against sensor-server collusion and Byzantine manipulation of participation. CRSF extends PG by introducing Byzantine-robust agreement on sensor submissions with threshold protection of circuit-input labels. Our work makes the following contributions:

1. We identify two limitations in PG’s trust assumptions: a single server unilaterally dictates sensor participation, and sensor-server collusion exposes secret circuit-input labels.
2. We present CRSF, a collusion-resilient garbled-circuit-based sensor-fusion protocol preserving the client-sensor-server workflow while resisting Byzantine participation manipulation and sensor-server collusion.
3. We introduce two protocol-level mechanisms for CRSF: agreement about sensor participation before status-dependent label release, and threshold protection of downstream circuit-input labels before fusion evaluation.
4. We prove that CRSF provides privacy against corrupted sensors colluding with one Byzantine server, and achieves correctness with explicit abort and liveness under our threat model.
5. We implement CRSF and evaluate its online performance on Google Cloud under both fault-free and representative faulty executions. The results show that the added sensor participation agreement and label reconstruction mechanisms incur moderate overhead, with total measured phase time remaining around one second at the largest tested instances.

*Paper organization.* Section 2 presents the system and threat model. Section 3 presents preliminaries and reviews the PG baseline. Section 4 analyzes the limitations of PG under our threat model. Section 5 describes the CRSF protocol. Section 6 proves its security. Section 7 presents the experimental evaluation. Section 8 discusses related work, and Section 9 concludes the paper.

## 2 System and Threat Model

**System Model.** We consider a sensor fusion system that involves three roles: a client, a set of  $n$  sensors ( $\{s_i\}_{i \in [0, n-1]}$ ) and four servers  $Ser_1, Ser_2, Ser_3, Ser_4$ . The client aims to obtain the output of a fusion function  $f$  evaluated over the private measurements of the sensors, without receiving those measurements in the clear. During the setup phase, the client may interact with the sensors and generate the cryptographic material required for the protocol. In the online phase, each sensor  $s_i$  locally encodes its measurement and sends the resulting encoded messages to the servers. The servers then perform the fusion computation on the encoded inputs and send the encoded outputs to the client, without learning the plaintext sensor measurements or the plaintext fusion result. During

this phase, the client communicates only with the servers and is the only party that can decode the final output. Sensors are mutually independent and do not communicate with each other.

**Communication Model.** We assume sensor-to-server links are synchronous with a known delay bound, so that a submission sent by an honest sensor reaches every honest server before the submission deadline  $T_{sub}$ . Inter-server communication is only assumed to be partially synchronous, as required by PBFT.

**Threat Model.** We consider a probabilistic polynomial-time adversary corrupting multiple sensors and at most one server. Corrupted sensors may arbitrarily deviate, submit malformed labels, and collude with the corrupted server. This server may exhibit Byzantine behaviors, such as sending inconsistent messages. The client honestly follows the protocol. The number of malicious sensors must remain below the underlying Marzullo-style algorithm’s fault threshold (e.g., fewer than  $1/3$  or  $1/2$  of all sensors [13]). Unavailable sensors also count toward this threshold.

**Relation to Existing Models.** This three-role abstraction follows a common pattern in sensing and aggregation systems [27, 8, 19]. The collusion threat considered here is also consistent with prior work on sensing and aggregation security: existing studies have shown that privacy and integrity guarantees may fail when a central service, aggregator, or server combines its view with corrupted data providers, users, or sensors [22, 26, 11, 20]. Our model instantiates this line of threat models in garbled-circuit-based sensor fusion, where sensor-server collusion creates additional risks for honest-sensor privacy.

**Security and Robustness Goals.** CRSF aims to achieve the following properties under the threat model considered in this work.

1. *Privacy.* Honest sensors’ private measurements remain hidden from every individual server, even under sensor-server collusion within the stated corruption bounds. The client learns only the protocol output and the protocol-visible status information needed for output delivery.
2. *Correctness.* An honest client either outputs  $\perp$  or accepts only the correct output of the fusion function evaluated on the protocol-determined sensor inputs and default replacements.
3. *Liveness.* An honest client eventually receives either the correct output or  $\perp$  under the stated assumptions.
4. *Byzantine-robust agreement on participation.* All honest servers agree on a common submission-level outcome for each sensor, namely whether to accept a submitted vector or exclude the sensor.

### 3 Preliminaries

**Garbled circuits.** Garbled circuits, introduced by Yao [28] and later formalized in simulation-based frameworks [15, 1], are a standard primitive for secure

computation. We use the standard syntax of a garbling scheme for Boolean circuits. A garbling scheme is a tuple of probabilistic polynomial-time algorithms  $\text{GC} = (\text{Gb}, \text{En}, \text{Ev}, \text{De})$ . On input a security parameter  $1^\kappa$  and a Boolean circuit  $C$ , the garbling algorithm samples  $(\hat{C}, \mathbf{e}, \mathbf{d}) \leftarrow \text{Gb}(1^\kappa, C)$ , where  $\hat{C}$  is the garbled circuit,  $\mathbf{e}$  is encoding information, and  $\mathbf{d}$  is decoding information. For a circuit with input wires  $w_1, \dots, w_m$ , a projective garbling scheme associates each input wire  $w_i$  with two labels  $L_{w_i}^0$  and  $L_{w_i}^1$ , representing the semantic values 0 and 1, respectively. Given an input vector  $x = (x_1, \dots, x_m) \in \{0, 1\}^m$ , the encoding algorithm outputs the encoded input vector  $X \leftarrow \text{En}(\mathbf{e}, x)$ , which contains exactly one label for each input wire. The evaluation algorithm computes  $Y \leftarrow \text{Ev}(\hat{C}, X)$ , where  $Y$  is a vector of garbled output labels. The decoding algorithm then maps these labels to a plaintext output,  $y \leftarrow \text{De}(\mathbf{d}, Y)$ . The decoded value satisfies  $y = C(x)$  except with negligible probability.

Garbling schemes are commonly studied with several security notions, including privacy, obliviousness, and authenticity [1]. Let  $\Phi(C)$  denote the public leakage associated with garbling  $C$ , such as the circuit size, input and output lengths, or topology, depending on the concrete scheme and application.

- *Obliviousness.* A garbling scheme  $\text{GC} = (\text{Gb}, \text{En}, \text{Ev}, \text{De})$  satisfies obliviousness if there exists a PPT simulator  $\text{Sim}_{\text{GC}}^{\text{obv}}$  such that, for every circuit  $C$  and input  $x$ ,  $(\hat{C}, X) \approx_c \text{Sim}_{\text{GC}}^{\text{obv}}(1^\kappa, \Phi(C))$ , where  $(\hat{C}, \mathbf{e}, \mathbf{d}) \leftarrow \text{Gb}(1^\kappa, C)$  and  $X \leftarrow \text{En}(\mathbf{e}, x)$ .
- *Authenticity.* A garbling scheme satisfies authenticity if, for every PPT adversary  $\mathcal{A}$ , every circuit  $C$ , and every input  $x$ , after  $(\hat{C}, \mathbf{e}, \mathbf{d}) \leftarrow \text{Gb}(1^\kappa, C)$ ,  $X \leftarrow \text{En}(\mathbf{e}, x)$ , and  $Y \leftarrow \text{Ev}(\hat{C}, X)$ , the probability that  $\mathcal{A}(1^\kappa, \hat{C}, X)$  outputs  $Y' \neq Y$  such that  $\text{De}(\mathbf{d}, Y') \neq \perp$  is negligible in  $\kappa$ .

**Shamir secret sharing.** A  $(t, n)$ -threshold Shamir secret sharing scheme [25] distributes a secret  $s \in \mathbb{F}$  among  $n$  parties over a finite field  $\mathbb{F}$ , where  $|\mathbb{F}| > n$ . The dealer samples a random polynomial  $p(X) = s + a_1X + \dots + a_{t-1}X^{t-1}$  of degree at most  $t - 1$ , where  $a_1, \dots, a_{t-1}$  are chosen independently and uniformly from  $\mathbb{F}$ . Each party  $i$  receives the share  $p(\alpha_i)$ , where  $\alpha_1, \dots, \alpha_n$  are distinct nonzero field elements. Any set of  $t$  shares can reconstruct  $s = p(0)$  by interpolation, whereas any set of fewer than  $t$  shares is statistically independent of  $s$ .

**Practical Byzantine Fault Tolerance.** Practical Byzantine Fault Tolerance (PBFT) [2] is a Byzantine fault-tolerant state-machine replication protocol. For  $n$  replicas, PBFT tolerates up to  $f$  Byzantine replicas provided that  $n \geq 3f + 1$ . In each consensus instance, a primary proposes a value in the pre-prepare phase, and replicas then use prepare and commit phases to form Byzantine quorums. PBFT provides safety and liveness under its standard assumptions. Safety requires that no two honest replicas decide different values in the same consensus instance. Liveness holds under partial synchrony, authenticated communication, and at most  $f$  Byzantine replicas. If the current primary is faulty or fails to make progress, honest replicas can initiate a view change, moving the protocol to a new view with a different primary while preserving previously prepared

information. Thus, once the network satisfies the synchrony condition, a faulty primary cannot indefinitely prevent progress. CRSF uses a PBFT-based submission agreement protocol rather than full state-machine replication. Specifically, the servers use PBFT’s quorum and phase structure to agree on an evidence-supported submission-level outcome for each sensor.

**The PG Baseline.** PG [13] is a privacy-preserving sensor-fusion protocol for the sensor-server-client architecture. It combines Marzullo-style fault-tolerant algorithms [18, 13] with garbled circuits. In addition to privacy-preserving computation, PG aims to tolerate malicious sensor behavior and to ensure output delivery under its original threat model.

At a high level, PG separates circuit garbling from sensor-input garbling: the client prepares the garbled circuits used by the protocol, while each sensor garbles its own private measurement into sensor-side input labels. The server thus receives the garbled circuits from the client and the garbled sensor inputs from the sensors, and then evaluates the relevant gates as in standard garbled-circuit evaluation. To support malformed-input detection and liveness, PG introduces two special gate types, called *checking gates* and *filter gates*. These gates form two input-processing layers that are evaluated by the server before it evaluates the garbled fusion circuit. For clarity, we use circuit-input labels to denote the labels used to evaluate the garbled fusion function circuit, distinguishing them from the labels used as inputs to the checking gates and filter gates. Note that garbled sensor inputs, namely sensor submissions, are not the circuit-input labels used for fusion circuit evaluation.

Checking gates are used to check whether the garbled sensor inputs submitted by sensors are well-formed. Each checking gate is associated with one or two specific input wires and is constructed so that any combination of valid labels for these wires is evaluated to  $0^\kappa$ . This allows the server to detect malformed sensor submissions from the outputs obtained by evaluating the checking gates, namely, a submission is malformed if the corresponding output is not a zero string. The server then informs the client which sensors are unavailable or fail to pass checking gate validation before proceeding to the filter gate layer.

Filter gates are then used to derive the circuit-input labels used in the subsequent fusion circuit evaluation. Each filter gate takes as input a sensor-side input label and a client-side label. The client releases the client-side label according to the server’s request, based on whether the corresponding sensor submitted well-formed labels. The filter gate is designed so that, for a sensor that submits well-formed labels, the server obtains circuit-input labels with the same truth values as the submitted sensor-side input labels. Otherwise, the server obtains a designated default label corresponding to the semantic value 1, which represents the maximum-value replacement for sensors that fail to submit well-formed labels. More concretely, consider a filter gate with two input wires, one for the client-side input and one for the sensor-side input.

Since PG inherits the FreeXOR [14] optimization, these circuit-input labels adopt a fixed global offset  $\Delta$ , satisfying  $W^1 = W^0 \oplus \Delta$ . The offset is used only for

wires in the fusion circuit, including these circuit-input labels, whereas checking gates and the input wires of filter gates are garbled independently of  $\Delta$ .

Section 4 analyzes the limitations of PG under the threat model of Section 2.

## 4 Limitations of PG

PG is secure under its original threat model, where the server does not collude with sensors and follows the prescribed participation procedure. Under the stronger model considered in this work, two limitations arise. The first requires active sensor-server collusion; the second is an inherent weakness of the PG design that a Byzantine server can exploit without any sensor cooperation.

First, PG does not preserve input privacy once sensor-server collusion is allowed. A Byzantine server colluding with a corrupted sensor can declare the sensor to be honest during filter-gate evaluation, causing the client to release the client-side label information for the honest branch. Since the corrupted sensor can generate both valid sensor-side input labels for its own input positions, the coalition can evaluate the corresponding filter gates under both labels and obtain both valid circuit-input labels on the same fusion-circuit input wire. XORing these labels reveals the FreeXOR offset  $\Delta$ . Once  $\Delta$  is known, any valid circuit-input label on a fusion-circuit wire reveals its complementary label. The adversary is therefore no longer restricted to the unique label assignment induced by the honest execution, which violates the intended input-privacy guarantee of PG. Also, knowing  $\Delta$  would trivially allow a malicious server to flip the semantic meaning of the fusion circuit outputs. Such output flipping additionally breaks the authenticity of the underlying garbling scheme: an adversary who knows  $\Delta$  can produce  $Y' \neq Y$  on the output wires such that  $\text{De}(d, Y')$  decodes to a valid but semantically flipped result, violating the standard authenticity guarantee that no PPT adversary should produce a non- $\perp$  decoded output different from  $C(x)$  [1].

Second, and independently of sensor-server collusion, PG relies on a single server to determine which sensor submissions are admitted to the fusion computation. A Byzantine server acting alone can therefore exclude honest sensors while retaining corrupted ones, changing the effective participating set and potentially violating the fault-threshold premise required by the underlying fault-tolerant fusion algorithm. This gap motivates the Byzantine-robust participation agreement goal introduced in this work (Section 2).

The two limitations have distinct structural roots. In the first case, the root weakness is that a colluding server can obtain both circuit-input labels of a wire by evaluating the filter gate under both sensor-input labels from a compromised sensor. In the second, it is that the single server in PG can unilaterally decide which sensor submissions are admitted.

*Illustrative example.* Consider a circuit with two input AND gates whose outputs feed a third AND gate. Sensor  $s_1$  provides the inputs to the first AND gate, and sensor  $s_2$  provides the inputs to the second. If  $s_1$  colludes with the server, the coalition can obtain both valid labels for  $s_1$ 's input wires and recover the global

FreeXOR offset  $\Delta$ . It can then derive complementary valid labels on other fusion-circuit wires, including those of  $s_2$ , and evaluate the second AND gate under alternative valid input-label combinations. Observing the resulting output labels reveals the semantic correspondence between  $s_2$ 's labels and their truth values. The output of the third AND gate can be flipped by XORing with  $\Delta$ . This illustrates why the original PG design does not provide privacy and correctness under sensor-server collusion.

These limitations motivate CRSF, which extends the PG-style garbled-circuit workflow while providing resilience against sensor-server collusion and Byzantine manipulation of sensor participation.

## 5 The CRSF Protocol

A seemingly direct way to mitigate the label-exposure problem in PG is to use OT-based label selection. In such a design, the client acts as the OT sender and provides each sensor with only the sensor-side input labels corresponding to that sensor's actual input bits. Thus, each sensor no longer obtains both valid sensor-side input labels for any input position, which prevents a sensor-server coalition from deriving both corresponding circuit-input labels on the same fusion-circuit input wire through filter-gate evaluation.

However, this approach has two limitations in our setting. Architecturally, it requires each sensor to run online OT with the client in addition to submitting encoded inputs to the servers. This increases the online cost for both sensors and the client, and forces the client to participate in per-sensor label-transfer interactions, whereas the client is intended to remain lightweight during the online phase. Functionally, OT-based label selection addresses only the label-exposure problem; it does not prevent a Byzantine server from manipulating which sensor submissions are admitted to the fusion computation.

CRSF therefore follows a different design. Its design is guided by two principles. First, an honest sensor submission delivered to the honest servers should not be unilaterally excluded by a Byzantine server, since such exclusion can invalidate the fault-tolerance guarantee of the protocol. Second, for each input wire of the fusion circuit, the servers should reconstruct only the circuit-input label determined by the agreed sensor submission and the accepted status vector. Accordingly, CRSF uses PBFT-based agreement among four servers to establish a common batch record for the current set of sensor submissions. CRSF also incorporates secret sharing into the filter gate design: each filter gate outputs a server-specific share rather than the complete circuit-input label.

We first present the setup assumptions and offline preparation in Section 5.1. The online protocol is then organized into two stages: Section 5.2 describes sensor submission collection and PBFT-based agreement, and Section 5.3 presents the subsequent online evaluation phase.

```

procedure PREPCRSF( $C_{\text{fus}}, \{\ell_i\}_{i \in [n]}$ ):
   $\Delta \leftarrow \{0, 1\}^{\kappa-1}$ 
  for each sensor  $s_i$  do
    establish seed  $K_i$  with  $s_i$ 
    for  $j \in \{0, \dots, \ell_i - 1\}$  and  $b \in \{0, 1\}$  do
       $\sigma_{i,j}^b \leftarrow \text{PRG}(K_i, i, j, b)$ 
    for each  $i, j, h$  do
      sample  $C_{i,j}^{\text{Hon},(h)}$  and  $C_{i,j}^{\text{Mal},(h)}$ 
    for each  $i, j$  do
       $(W_{i,j}^0, W_{i,j}^1, \{W_{i,j}^{b,(h)}\}_{b,h})$ 
       $\leftarrow \text{SHAREWIRE}(\Delta)$ 
    for each  $i, j, h$  do
       $\hat{F}_{i,j}^h \leftarrow \text{GBFILT}(C_{i,j}^{\text{Hon},(h)}, C_{i,j}^{\text{Mal},(h)},$ 
         $\sigma_{i,j}^0, \sigma_{i,j}^1, W_{i,j}^0, W_{i,j}^1, W_{i,j}^{1,(h)})$ 
    for each  $s_i$  and  $t \in \{0, \dots, \lceil \ell_i/2 \rceil - 1\}$  do
       $j \leftarrow 2t$ 
      if  $j + 1 < \ell_i$  then
         $\hat{G}_{i,t}^{(1)} \leftarrow \text{GBCHK}(\sigma_{i,j}^0, \sigma_{i,j}^1, \sigma_{i,j+1}^0, \sigma_{i,j+1}^1)$ 
         $\hat{G}_{i,t}^{(2)} \leftarrow$ 
           $\text{GBCHK}(W_{i,j}^0, W_{i,j}^1, W_{i,j+1}^0, W_{i,j+1}^1)$ 
      else
         $\hat{G}_{i,t}^{(1)} \leftarrow \text{GBCHK1}(\sigma_{i,j}^0, \sigma_{i,j}^1)$ 
         $\hat{G}_{i,t}^{(2)} \leftarrow \text{GBCHK1}(W_{i,j}^0, W_{i,j}^1)$ 
       $\hat{C}_{\text{fus}} \leftarrow \text{Gb}(C_{\text{fus}}, \Delta, \{W_{i,j}^0, W_{i,j}^1\}_{i,j})$ 
      send  $\hat{C}_{\text{fus}}$  and  $\{\hat{G}_{i,t}^{(1)}, \hat{G}_{i,t}^{(2)}\}_{i,t}$  to all servers
    for each server  $Ser_h$  do
      send  $\{\hat{F}_{i,j}^h\}_{i,j}$  to  $Ser_h$ 

procedure SHAREWIRE( $\Delta$ ):
  sample  $W^0 \leftarrow \{0, 1\}^\kappa$ 
  set  $W^1 \leftarrow W^0 \oplus \Delta$ 
  for  $b \in \{0, 1\}$  do
     $(W^{b,(1)}, W^{b,(2)}, W^{b,(3)}, W^{b,(4)})$ 
     $\leftarrow \text{Share}_{3,4}^{\text{byte}}(W^b)$ 
  return  $(W^0, W^1, \{W^{b,(h)}\}_{b,h})$ 

procedure GBFILT( $C^{\text{Hon}}, C^{\text{Mal}}, S^0, S^1, \omega^0, \omega^1$ ):
  define mapping  $\mathcal{T}_F$  by
     $\mathcal{T}_F(C^{\text{Hon}}, S^0) = \omega^0$ 
     $\mathcal{T}_F(C^{\text{Hon}}, S^1) = \omega^1$ 
     $\mathcal{T}_F(C^{\text{Mal}}, \emptyset) = \omega^1$ 
  return  $\text{GbGate}(\mathcal{T}_F)$ 

procedure GBCHK( $A^0, A^1, B^0, B^1$ ):
  define mapping  $\mathcal{T}_G$  by
    for  $b, b' \in \{0, 1\}$  do
       $\mathcal{T}_G(A^b, B^{b'}) = 0^\kappa$ 
  return  $\text{GbGate}(\mathcal{T}_G)$ 

procedure GBCHK1( $A^0, A^1$ ):
  define mapping  $\mathcal{T}_{G1}$  by
     $\mathcal{T}_{G1}(A^0) = 0^\kappa$ 
     $\mathcal{T}_{G1}(A^1) = 0^\kappa$ 
  return  $\text{GbGate}(\mathcal{T}_{G1})$ 

```

**Fig. 1.** CRSF offline preparation. The procedure GbGate garbles a special gate according to the specified input-label-to-output-label mapping.

## 5.1 Setup and Offline Preparation

**Setup and authentication material.** The protocol uses a collision-resistant hash function  $H : \{0, 1\}^* \rightarrow \{0, 1\}^\kappa$  [23], an EUF-CMA secure digital signature scheme  $\text{Sig} = (\text{KeyGen}, \text{Sign}, \text{Verify})$  [10], and a keyed pseudorandom label-derivation function PRG. Each sensor  $s_i$  holds a signing key  $\text{sk}_{s_i}$ , and its verification key  $\text{vk}_{s_i}$  is known to all servers. Each server  $Ser_h$  holds a signing key  $\text{sk}_{Ser_h}$ , and its verification key  $\text{vk}_{Ser_h}$  is known to the client and the other servers. Each sensor also shares a pairwise secret key with the client for deriving sensor-side input labels.

**Offline preparation.** The client prepares the garbled tables and label material required for the online protocol (detailed in Fig. 1). It distributes the garbled fusion circuit and checking gates to all servers, but each server receives only its server-specific filter gates.

## 5.2 Agreement on Sensor Submissions

Online phase of CRSF starts with a PBFT-based evidence-supported agreement phase among the four servers. This phase establishes a common submission-level

record for the current session. For clarity, we describe the agreement rule for an individual sensor submission. For each sensor  $s_i$ , the servers determine a common submission-level outcome  $o_i \in \{(\text{Hon}, r_i), \text{Mal}\}$  before label validation and fusion evaluation. The labels **Hon** and **Mal** denote protocol-level acceptance states: **Hon** means that the servers agree to use  $r_i$  as the submitted vector for  $s_i$ , whereas **Mal** means that  $s_i$  is excluded from the current session. In implementation, the servers batch the agreement for all sensors in the same session by agreeing on an outcome vector  $O = (o_1, \dots, o_n)$ . Let  $f = 1$ . The four-server setting tolerates one Byzantine server, and each PBFT quorum has size  $2f + 1 = 3$ .

**Sensor-side submission.** Let  $\text{sid}$  be a session identifier. For each target server  $Ser_h$ , let  $r_i^{(h)}$  be the complete input-label vector submitted by  $s_i$  to  $Ser_h$ , and let  $d_i^{(h)} = H(r_i^{(h)})$ . The sensor signs the session identifier, the sensor identity, the target server identity, and the digest as  $\pi_i^{(h)} = \text{Sign}_{\text{sk}_{s_i}}(\text{sid}, i, h, d_i^{(h)})$ . The submission sent to  $Ser_h$  is  $\text{sub}_i^{(h)} = (r_i^{(h)}, \pi_i^{(h)})$ .

**Server-side agreement.**

1. **Local reports.** After the sensor-submission deadline  $T_{\text{sub}}$ , each server forms one signed local report for  $s_i$ . If  $Ser_h$  received a valid submission before  $T_{\text{sub}}$ , it creates a submission report  $R_i^{(h)} = (\text{Sub}, h, \text{sub}_i^{(h)}, \rho_i^{(h)})$ , where  $\rho_i^{(h)} = \text{Sign}_{\text{sk}_{Ser_h}}(\text{sid}, i, h, \text{Sub}, H(\text{sub}_i^{(h)}))$ . If  $Ser_h$  did not receive a valid submission before  $T_{\text{sub}}$ , it creates a no-submission report  $R_i^{(h)} = (\text{NoSub}, h, T_{\text{sub}}, \rho_i^{(h)})$ , where  $\rho_i^{(h)} = \text{Sign}_{\text{sk}_{Ser_h}}(\text{sid}, i, h, \text{NoSub}, T_{\text{sub}})$ . The primary also forms its own local report, and each non-primary server sends its local report to the primary of the current view. The view identifies the current primary for this agreement instance.
2. **Evidence rule.** The primary constructs an evidence set  $E_i$  consisting of  $2f + 1 = 3$  valid reports from distinct servers. A report is valid if the server signature verifies, the server identity is distinct within  $E_i$ , and every embedded sensor submission verifies under  $\text{vk}_{s_i}$ . The outcome is derived deterministically from  $E_i$ . If  $E_i$  contains at least  $f + 1 = 2$  valid submission reports with the same input-label vector  $r_i$ , then the only valid outcome is  $o_i = (\text{Hon}, r_i)$ . Otherwise, the derived outcome is  $o_i = \text{Mal}$ .
3. **PBFT-based agreement.**
  - *Pre-prepare.* The primary broadcasts a pre-prepare proposal  $\langle \text{PRE-PREPARE}, \text{sid}, i, \text{view}, o_i, E_i \rangle_{Ser_p}$ . A server accepts a pre-prepare message only if the primary signature verifies, the instance identifier is correct,  $E_i$  is a valid evidence set, and  $o_i$  is exactly the outcome derived from  $E_i$ . Thus, a proposal for **Mal** is invalid if  $E_i$  contains two matching valid submissions, and a proposal for  $(\text{Hon}, r_i)$  is invalid if  $E_i$  does not contain two matching valid submissions for  $r_i$ .
  - *Prepare and commit.* After accepting a valid pre-prepare proposal, a server broadcasts a prepare message for the digest  $H(o_i, E_i)$ . A server becomes prepared for this digest after it gets three matching prepare messages from

distinct servers, including the pre-prepare message from the primary. Once prepared, the server broadcasts a commit message for the same digest. It decides  $o_i$  only after obtaining three matching commit messages for this digest.

If the primary fails to send a valid pre-prepare message before timeout, or if equivocation prevents a prepare quorum, backups invoke a view-change procedure. The new primary preserves any prepared value required by PBFT safety. Otherwise, it constructs a fresh evidence set from three distinct servers' reports and proposes the deterministically derived outcome.

Under PBFT safety, all honest servers decide the same evidence-supported outcome for each sensor: either  $(\text{Hon}, r_i)$  for a common accepted input-label vector, or  $\text{Mal}$  for exclusion. In addition, the evidence rule prevents a delivered honest submission from being excluded. Under the sensor-delivery assumption, if  $s_i$  is honest and participates, then all honest servers receive the same valid submission from  $s_i$  before  $T_{\text{sub}}$ . Since at most one server is Byzantine, every evidence set of three distinct valid reports contains at least two honest reports, and these reports carry the same submission vector and digest. Therefore, the deterministic evidence rule derives  $o_i = (\text{Hon}, r_i)$ , rather than  $\text{Mal}$ .

### 5.3 Status-Dependent Label Release and Reconstruction

Following the PBFT-based agreement in Section 5.2, the servers hold a common outcome vector  $O = (o_1, \dots, o_n)$ . This vector identifies accepted submissions, but does not validate whether the included sensor-side input labels are well-formed. The first-layer checking gates perform this validation, yielding a status vector  $u = (u_1, \dots, u_n)$ . Each entry determines whether the client releases the  $\text{Hon}$  or  $\text{Mal}$  branch of the corresponding client-side filter-gate labels. The remainder of this section details the online evaluation phase: status derivation, server-specific label release, reconstruction, and final fusion-circuit evaluation.

#### Online Evaluation.

1. **First-layer checking.** For each sensor  $s_i$ , if  $o_i = (\text{Hon}, r_i)$ , each server evaluates the first-layer checking gates on the sensor-side input labels in  $r_i$ . If all checking-gate outputs are  $0^\kappa$ , the status of  $s_i$  is set to  $\text{Hon}$ ; otherwise, it is set to  $\text{Mal}$ . Thus, each server derives a status vector  $u = (u_1, \dots, u_n)$  according to

$$u_i = \begin{cases} \text{Hon}, & \text{if } o_i = (\text{Hon}, r_i) \text{ and } r_i \text{ passes the first-layer checking gates,} \\ \text{Mal}, & \text{otherwise.} \end{cases}$$

2. **Filter gate transformation.**

- *Label release.* Each server sends the resulting status vector  $u = (u_1, \dots, u_n)$  to the client. The client accepts a status vector only if at least three servers report the same vector; otherwise, the client aborts. Let  $u^*$  denote this accepted majority status vector. Once such a 3-of-4 majority status vector is obtained, the client sends to each server its server-specific client-side

filter-gate labels corresponding to this majority vector. In particular, for each sensor and input position, the client releases only one branch of client-side labels and never releases both the **Hon** and **Mal** branch labels.

- *Evaluation.* The servers then enter the filter-gate layer. For each server  $Ser_h$ , each sensor  $s_i$ , and each input position  $j$ , server  $Ser_h$  evaluates its server-specific filter gate using the released client-side label and the corresponding sensor-side input label if  $u_i^* = \text{Hon}$ , or using the **Mal** branch otherwise. The output is not a complete circuit-input label. Instead, it is a server-specific share of the circuit-input label determined by the accepted status branch and the corresponding sensor-side input label.
- 3. **Reconstruction with checking.** The servers exchange these shares and reconstruct candidate circuit-input labels from triples of shares. Since each complete circuit-input label is shared using byte-wise  $(3, 4)$  Shamir sharing, any three valid shares determine one candidate label. A server accepts a reconstructed candidate label set only if it passes the check of another layer checking gates, which we call the second-layer checking gates. If no candidate passes the check, the server aborts. All the checked and validated circuit-input label sets are used as the input to the fusion circuit evaluation.
- 4. **Fusion circuit evaluation.** Finally, each server evaluates the garbled fusion circuit on the validated circuit-input labels and obtains an output-label vector. The servers send their output-label vectors to the client.

The client accepts an output-label vector only if it receives the same vector from at least three servers; otherwise, it aborts. It then decodes the accepted vector using its output decoding information and outputs the fusion result. This completes the online execution of CRSF.

## 6 Security Analysis

We now analyze the security guarantees provided by CRSF under the adversarial model of Section 2. We assume that the client honestly generates the setup material and that public verification keys are distributed correctly. Cryptographically, we assume that  $H$  is collision resistant,  $\text{Sig}$  is EUF-CMA secure,  $\text{PRG}$  is pseudorandom, the garbling scheme satisfies privacy, obliviousness, and authenticity, and the byte-wise  $(3, 4)$  Shamir sharing scheme satisfies correctness and privacy. For liveness, we additionally assume the standard PBFT liveness conditions, including authenticated communication, partial synchrony, and eventual delivery of messages among honest parties.

We begin by defining the corrupted coalition’s view and the leakage available to the simulator, which are then used to state and prove the privacy theorem. We then establish correctness with explicit abort and liveness.

Let  $\hat{C}_{\text{fus}}$  denote the common garbled fusion circuit, and let  $\hat{\mathcal{G}}^{(1)}$  and  $\hat{\mathcal{G}}^{(2)}$  denote the common first- and second-layer checking gates. All these common gates are known to all the servers. For each server  $Ser_h$ , let  $\hat{\mathcal{F}}_h$  denote its server-specific filter gates. We write  $\hat{\mathcal{F}}_q$  for the filter gates in the corrupted server’s view. Let  $\Phi_{\text{fus}}$  denote the public structural leakage of the garbled fusion circuit,

such as its topology, size, input length, and output length. Let  $\Phi_{\text{aux}}$  denote the public structural leakage of the checking and filter layers, including the number and arity of checking gates and filter gates, the input lengths  $\{\ell_i\}_{i \in [n]}$ , and the public wiring pattern between these auxiliary gates and the fusion-circuit input wires.

Fix an adversary  $\mathcal{A}$  that corrupts server  $Ser_q$  and a set of sensors. Let  $\text{View}_{\mathcal{A},q}$  denote the joint view of the corrupted coalition. This view consists of the corrupted parties' inputs, randomness, and local states; the corrupted sensors' submitted labels; the messages sent and received by  $Ser_q$ ; the PBFT-based agreement transcript visible to  $Ser_q$ ; the status-vector and client-side label-release transcript; the garbled artifacts visible to  $Ser_q$ , namely  $\hat{C}_{\text{fus}}$ ,  $\hat{G}^{(1)}$ ,  $\hat{G}^{(2)}$ , and  $\hat{F}_q$ ; the checking- and filter-gate outputs obtained by  $Ser_q$ ; the reconstruction shares received by  $Ser_q$ ; the reconstructed circuit-input labels visible to  $Ser_q$ ; and, if the protocol reaches the final evaluation phase, the output-label vector obtained from evaluating the garbled fusion circuit. The client-side decoding information is not included in  $\text{View}_{\mathcal{A},q}$ .

**Definition 1 (Coalition leakage).** *For an adversary  $\mathcal{A}$  corrupting server  $Ser_q$  and a set of sensors, define*

$$L_{\mathcal{A},q}^{\text{coal}} = (q, \text{inp}_{\mathcal{A},q}, \text{rand}_{\mathcal{A},q}, \Phi_{\text{fus}}, \Phi_{\text{aux}}, \text{type}(O), u^*, \text{abortinfo}_q).$$

Here  $\text{inp}_{\mathcal{A},q}$  and  $\text{rand}_{\mathcal{A},q}$  are the corrupted parties' inputs and randomness,  $\text{type}(O)$  keeps only the Hon/Mal type of each agreed outcome and removes every accepted vector  $r_i$ , and  $\text{abortinfo}_q$  records the public abort point and visible transcript shape before abort.

Let  $\text{pp}$  denote the public parameters generated during setup, including public verification keys, public cryptographic descriptions, session identifiers, and public structural information.

**Theorem 1 (Privacy).** *Under the stated adversarial, setup, and cryptographic assumptions, for every PPT adversary  $\mathcal{A}$  corrupting one server  $Ser_q$  and any set of corrupted sensors within the fault threshold tolerated by the underlying fusion algorithm, there exists a PPT simulator  $\mathcal{S}$  such that*

$$(\text{pp}, \text{View}_{\mathcal{A},q}) \approx_c \mathcal{S}(1^\kappa, L_{\mathcal{A},q}^{\text{coal}}).$$

Hence, the corrupted coalition learns neither honest sensors' measurements nor the plaintext fusion output beyond the coalition leakage  $L_{\mathcal{A},q}^{\text{coal}}$ .

*Proof (sketch).* The proof is by simulation. Given  $L_{\mathcal{A},q}^{\text{coal}}$ , the simulator constructs a simulated outcome vector  $\tilde{O}$  with  $\text{type}(\tilde{O}) = \text{type}(O)$ . For honest-sensor Hon entries, it samples simulated submitted vectors  $\tilde{r}_i$  from the pseudorandom label distribution and uses them consistently in the simulated submissions, reports, evidence sets, first-layer checking gates, and filter-gate inputs; corrupted sensors and the corrupted server generate their messages by running  $\mathcal{A}$ . The PBFT-based transcript is generated from this simulated submission-report-evidence

chain, with prepare/commit messages bound to  $H(\tilde{o}_i, \tilde{E}_i)$ . The simulator then invokes the garbled-circuit obliviousness simulator only for the final garbled fusion circuit, obtaining  $(\tilde{C}_{\text{fus}}, \tilde{X}_{\text{fus}}, \tilde{Y}_{\text{fus}})$ , and coherently regenerates the filter-gate outputs, reconstruction shares, second-layer checking gates, and output-label messages so that reconstruction yields exactly  $\tilde{X}_{\text{fus}}$ . For complementary circuit-input labels, the simulator replaces the coalition's visible shares with independently generated uniform field-element strings of the same form. Indistinguishability follows from the privacy guarantee of Shamir secret sharing, PRG security for honest sensor-side labels, garbled-circuit obliviousness for the final fusion circuit, signature unforgeability, collision resistance, and garbled-gate authenticity. Hence the simulated view is computationally indistinguishable from  $(\text{pp}, \text{View}_{\mathcal{A},q})$ .  $\square$

Appendix A gives the full simulation proof for Theorem 1.

**Theorem 2 (Correctness with Explicit Abort).** *Under the stated adversarial and cryptographic assumptions, the honest client either outputs  $\perp$  or accepts the correct output-label vector for the uniquely determined circuit-input label vector, except with negligible probability.*

*Proof.* PBFT safety and the deterministic evidence rule imply that all honest servers that continue the protocol use the same outcome vector  $O$ . The first-layer checking gates are deterministic, so the honest servers derive the same final status vector. Since the client releases filter-gate labels only after receiving a three-server majority on a status vector, and at most one server is Byzantine, the accepted vector  $u^*$  equals the honest servers' final status vector. Therefore, all honest servers evaluate the filter gates according to the same status vector and generate shares for the same uniquely determined circuit-input label vector. The reconstruction procedure either recovers this vector and validates it through the second-layer checking gates, or aborts. Once reconstruction succeeds, all honest servers evaluate the same final garbled circuit on the same input-label vector. A Byzantine server cannot create a different three-server majority, and garbled-circuit authenticity prevents the client from accepting an invalid output-label vector except with negligible probability.  $\square$

**Theorem 3 (Liveness).** *Under the stated adversarial assumptions and PBFT liveness conditions, the honest client eventually outputs either a decoded fusion result or  $\perp$ .*

*Proof.* PBFT liveness ensures that each sensor-submission agreement instance eventually decides an evidence-supported outcome, unless an explicit abort condition is triggered. If an abort is triggered, the client outputs  $\perp$ . Otherwise, after all agreement instances terminate, the remaining phases consist of finite local computations and finite message exchanges: first-layer checking, status reporting, client-side label release, filter-gate evaluation, share exchange, reconstruction, second-layer checking, final garbled-circuit evaluation, and output submission.

Each phase either completes or triggers an explicit abort. If no abort is triggered, the client eventually receives three matching output-label vectors from the honest servers and decodes the accepted vector. Hence, the client eventually outputs either the decoded fusion result or  $\perp$ .  $\square$

## 7 Evaluation

We implemented CRSF in C++. Following PG [13], we evaluate five Marzullo-style fault-tolerant fusion circuits in which each sensor contributes a 16-bit value, and scale the number of sensors from 3 to at most 261. We used the same circuit files from the open source repository of PG<sup>7</sup>; for more detailed descriptions of these algorithms, please refer to [13]. For authenticated sensor submissions and inter-server PBFT messages, our implementation uses SHA-256 for message digests and Ed25519 detached signatures via libsodium. Concretely, we first compute a SHA-256 digest of each message and then generate or verify an Ed25519 detached signature on that digest.

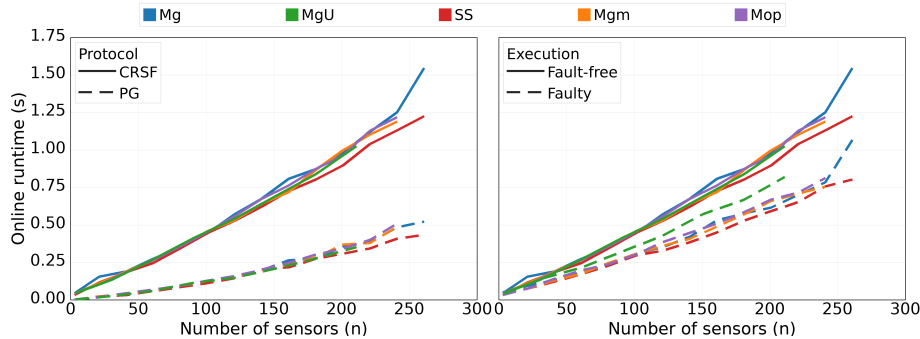
All distributed measurements were obtained on Google Cloud Platform in region `us-central1`, zone `us-central1-a` (Iowa). Each virtual machine is an `e2-standard-2` instance with 2 vCPUs, 8 GiB RAM, and a 40 GB balanced persistent disk, running Ubuntu 22.04 LTS. CRSF experiments use five VMs: four server nodes and one client-side node hosting the garbler and software-emulated sensor processes. The PG baseline uses two VMs, one for the client-side processes and one for the single server. All parties communicate over the VPC network within the same zone.

Our experiments use software-emulated sensors rather than physical sensing devices. Each emulated sensor produces the same protocol-level object as a real sensor in CRSF: a 16-bit input value and an authenticated submission. The evaluation therefore measures the online cost of the CRSF protocol execution, including authenticated submission processing, PBFT-based agreement, label reconstruction, and garbled-circuit evaluation. It does not model physical sensor acquisition time, wireless-link variability, or embedded-device energy consumption. All reported runtimes exclude the offline preparation phase.

The left plot of Fig. 2 compares the online runtime of PG and CRSF as the number of sensors increases. Across all five fusion circuits, PG remains faster because it uses a single evaluator and does not incur the inter-server PBFT agreement and threshold share reconstruction steps introduced by CRSF. For a fixed four-server deployment, the runtime of CRSF grows approximately linearly with the number of sensors, indicating predictable scaling despite these additional robustness mechanisms.

The right plot of Fig. 2 compares CRSF under fault-free and faulty executions. In the evaluated faulty setting, we instantiate the maximum tolerated sensor faults by having the corresponding faulty sensors omit their submissions, and instantiate one server fault by making one backup server non-responsive.

<sup>7</sup> <https://doi.org/10.6084/m9.figshare.25669026.v2>



**Fig. 2.** Online runtime of PG and CRSF under different execution settings. Colors denote the fusion algorithms. In the left plot, solid and dashed lines denote CRSF and PG, respectively. In the right plot, solid and dashed lines denote fault-free and faulty CRSF executions, respectively.

After the non-responsive backup is detected, the primary proceeds with the remaining responsive servers, which are sufficient to form the required quorum. In this setting, the responsive servers process fewer sensor-submission records and perform fewer signature verifications during the agreement and validation steps. This explains why the measured online runtime can be lower than in the corresponding fault-free execution. This faulty execution represents one evaluated adversarial scenario rather than an exhaustive fault-injection campaign. Overall, CRSF trades higher online runtime for Byzantine-resilient distributed evaluation and robustness against sensor-server collusion.

Table 1 further breaks down the main CRSF-specific online costs at the largest tested instance for each fusion algorithm. The PBFT-based submission agreement accounts for the largest communication volume, ranging from about 7.8–9.7 MB in fault-free executions, because each server exchanges authenticated evidence for batched sensor submissions. Threshold share reconstruction is smaller in communication volume, about 1.2–1.5 MB in the fault-free cases, but still contributes measurable wall time because the servers exchange and validate shares before evaluating the garbled fusion circuit. Across the five algorithms, the measured wall time of the BFT agreement phase is below 0.51 seconds, and the reconstruction phase is below 0.14 seconds in the fault-free setting. Faulty executions have lower communication volume and wall time in this table, which is consistent with the performance trend shown in Fig. 2.

## 8 Related Work

**Collusion-resilient sensing and aggregation.** Prior work has studied collusion resistance in several sensing and aggregation settings. Shi et al. [26] propose PriSense, a privacy-preserving aggregation scheme for people-centric urban sensing based on data slicing and mixing. PriSense supports additive and

**Table 1.** Communication volume and measured phase wall time at the largest tested circuit per algorithm.

| Circuit | Scenario   | $n$ | BFT agreement |       | Reconstruction |       | Total  |        |
|---------|------------|-----|---------------|-------|----------------|-------|--------|--------|
|         |            |     | KB            | ms    | KB             | ms    | KB     | ms     |
| Mg      | Fault-free | 261 | 9891          | 488.0 | 1566           | 115.7 | 15 492 | 1546.0 |
|         | Faulty     |     | 8365          | 474.9 | 783            | 39.0  | 12 424 | 1066.3 |
| MgU     | Fault-free | 211 | 7998          | 426.4 | 1266           | 122.9 | 12 526 | 1025.2 |
|         | Faulty     |     | 6861          | 369.8 | 633            | 39.9  | 10 181 | 819.4  |
| SS      | Fault-free | 261 | 9891          | 506.5 | 1566           | 138.2 | 15 492 | 1224.2 |
|         | Faulty     |     | 8365          | 353.0 | 783            | 47.5  | 12 424 | 802.2  |
| Mgm     | Fault-free | 241 | 9134          | 481.7 | 1446           | 128.6 | 14 305 | 1189.7 |
|         | Faulty     |     | 7725          | 323.5 | 723            | 34.3  | 11 473 | 754.9  |
| Mop     | Fault-free | 241 | 9134          | 472.3 | 1446           | 139.3 | 14 305 | 1217.8 |
|         | Faulty     |     | 7725          | 325.9 | 723            | 45.7  | 11 473 | 813.1  |

non-additive statistical queries and protects user privacy against a threshold number of colluding users and aggregation servers. Günther et al. [11] revisit the PEPSI participatory-sensing model and show that cross-party collusion, such as collusion between a mobile node and a querier or between the service provider and a mobile node, can break earlier privacy guarantees. Their PEP-SICo construction adds collusion resistance and supports optional aggregation for small sensor readings using an additively homomorphic identity-based encryption scheme. Yao et al. [29] study privacy-preserving SUM aggregation in two-tiered mobile wireless sensor networks and propose PDAAS and PDACAS. In particular, PDACAS protects the raw readings of mobile sensor nodes even when the cell header, acting as the aggregator, and the sink collude, while still allowing the sink to derive the SUM. These works motivate the relevance of collusion in sensing systems, but they differ from CRSF in both functionality and leakage surface. PriSense and PDACAS are privacy-preserving aggregation protocols: they hide individual users’ or sensors’ readings while enabling an aggregation server or sink to compute the intended aggregate. PEPSICo addresses a different participatory-sensing problem. Complementary work studies secure and verifiable aggregation against malicious aggregators and corrupted users or sensors [22, 20]. These works address aggregation correctness or verifiable summation, whereas CRSF targets garbled-circuit-based sensor fusion with sensor-server collusion and Byzantine-robust participation agreement.

**Secure aggregation and federated learning.** Our threat model and technical setting are fundamentally different from those considered in secure aggregation for federated learning [9, 21, 31, 32]. Classical secure aggregation protocols are designed to hide each client’s local model update while allowing the server to recover an aggregate. These formulations usually involve only two roles, namely clients and a server, and the aggregate is intentionally revealed to the server. For

example, efficient dropout-resilient aggregation [17] proposes a scalable protocol for secure sum computation, where secret sharing is used primarily to recover masks under client dropouts, and the server learns the final aggregate. FLock [3] instead adopts a more MPC-style design with multiple aggregators, enabling richer aggregation functionalities at the cost of higher system complexity. In contrast, CRSF targets three-role sensor fusion, where the servers should learn neither individual sensor measurements nor the final fusion result, even under sensor-server collusion and Byzantine server behavior.

## 9 Conclusion

We presented CRSF, a collusion-resilient privacy-preserving sensor-fusion protocol for the sensor-server-client architecture. It combines PBFT-based submission agreement with status-dependent label release and byte-wise  $(3, 4)$  Shamir sharing of circuit-input labels, preventing a Byzantine server from unilaterally controlling participation and preventing an admissible sensor-server coalition from reconstructing complementary circuit-input labels. We proved privacy, correctness with explicit abort, and liveness under the stated assumptions. Our implementation and Google Cloud evaluation show that CRSF introduces additional online runtime from agreement and reconstruction, but retains predictable scaling across the evaluated fault-tolerant fusion circuits. These results show that stronger robustness against sensor-server collusion and Byzantine participation manipulation can be achieved with moderate online overhead.

## Acknowledgment

Chenglu Jin is (partially) supported by project CiCS of the research programme Gravitation, which is (partly) financed by the Dutch Research Council (NWO) under the grant 024.006.037. Chao Yin is supported by the China Scholarship Council (CSC) and the Dutch Sector Plan. This work is supported in part by the National Natural Science Foundation of China under 62272043 and Yangtze Delta Region Institute of Tsinghua University, Zhejiang (No.LZZLX24F007).

## References

1. Bellare, M., Hoang, V.T., Rogaway, P.: Foundations of garbled circuits. In: Proceedings of the 2012 ACM conference on Computer and communications security. pp. 784–796 (2012)
2. Castro, M., Liskov, B.: Practical byzantine fault tolerance and proactive recovery. *ACM Transactions on Computer Systems (TOCS)* **20**(4), 398–461 (2002)
3. Chen, R., Dong, Y., Liu, Y., Fan, T., Li, D., Guan, Z., Liu, J., Zhou, J.: Flock: Robust and privacy-preserving federated learning based on practical blockchain state channels. In: Proceedings of the ACM on Web Conference 2025. pp. 884–895 (2025)

4. Conti, M., Zhang, L., Roy, S., Di Pietro, R., Jajodia, S., Mancini, L.V.: Privacy-preserving robust data aggregation in wireless sensor networks. *Security and Communication Networks* **2**(2), 195–213 (2009)
5. Cornelius, C., Kapadia, A., Kotz, D., Peebles, D., Shin, M., Triandopoulos, N.: Anonymsense: privacy-aware people-centric sensing. In: *Proceedings of the 6th international conference on Mobile systems, applications, and services*. pp. 211–224 (2008)
6. De Cristofaro, E., Ding, X., Tsudik, G.: Privacy-preserving querying in sensor networks. *arXiv preprint arXiv:0905.2158* (2009)
7. De Cristofaro, E., Soriente, C.: Short paper: Pepsi—privacy-enhanced participatory sensing infrastructure. In: *Proceedings of the fourth ACM conference on Wireless network security*. pp. 23–28 (2011)
8. De Cristofaro, E., Soriente, C.: Participatory privacy: Enabling privacy in participatory sensing. *IEEE network* **27**(1), 32–36 (2013)
9. Fereidooni, H., Marchal, S., Miettinen, M., Mirhoseini, A., Möllering, H., Nguyen, T.D., Rieger, P., Sadeghi, A.R., Schneider, T., Yalame, H., et al.: Safelearn: Secure aggregation for private federated learning. In: *2021 IEEE security and privacy workshops (SPW)*. pp. 56–62. IEEE (2021)
10. Goldwasser, S., Micali, S., Rivest, R.L.: A digital signature scheme secure against adaptive chosen-message attacks. *SIAM Journal on computing* **17**(2), 281–308 (1988)
11. Günther, F., Manulis, M., Peter, A.: Privacy-enhanced participatory sensing with collusion resistance and data aggregation. In: *International Conference on Cryptology and Network Security*. pp. 321–336. Springer (2014)
12. He, W., Liu, X., Nguyen, H., Nahrstedt, K., Abdelzaher, T.: Pda: Privacy-preserving data aggregation in wireless sensor networks. In: *IEEE INFOCOM 2007-26th IEEE International Conference on Computer Communications*. pp. 2045–2053. IEEE (2007)
13. Jin, C., Yin, C., Van Dijk, M., Duan, S., Massacci, F., Reiter, M.K., Zhang, H.: PG: Byzantine fault-tolerant and privacy-preserving sensor fusion with guaranteed output delivery. In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. pp. 3272–3286 (2024)
14. Kolesnikov, V., Schneider, T.: Improved garbled circuit: Free xor gates and applications. In: *International Colloquium on Automata, Languages, and Programming*. pp. 486–498. Springer (2008)
15. Lindell, Y., Pinkas, B.: A proof of security of yao’s protocol for two-party computation. *Journal of cryptology* **22**(2), 161–188 (2009)
16. Liu, J., Bic, L., Gong, H., Zhan, S.: Data collection for mobile crowdsensing in the presence of selfishness. *EURASIP journal on wireless communications and networking* **2016**(1), 82 (2016)
17. Liu, Z., Guo, J., Lam, K.Y., Zhao, J.: Efficient dropout-resilient aggregation for privacy-preserving machine learning. *IEEE Transactions on Information Forensics and Security* **18**, 1839–1854 (2022)
18. Marzullo, K.: Tolerating failures of continuous-valued sensors. *ACM Transactions on Computer Systems (TOCS)* **8**(4), 284–304 (1990)
19. Nath, S., Yu, H., Chan, H.: Secure outsourced aggregation via one-way chains. In: *Proceedings of the 2009 ACM SIGMOD International Conference on Management of data*. pp. 31–44 (2009)
20. Palazzo, M., Dekker, F.W., Brighente, A., Conti, M., Erkin, Z.: Privacy-preserving data aggregation with public verifiability against internal adversaries. In: *33rd USENIX Security Symposium (USENIX Security 24)*. pp. 6957–6974 (2024)

21. Pasquini, D., Francati, D., Ateniese, G.: Eluding secure aggregation in federated learning via model inconsistency. In: Proceedings of the 2022 ACM SIGSAC Conference on computer and communications security. pp. 2429–2443 (2022)
22. Przydatek, B., Song, D., Perrig, A.: Sia: Secure information aggregation in sensor networks. In: Proceedings of the 1st international conference on Embedded networked sensor systems. pp. 255–265 (2003)
23. Rogaway, P., Shrimpton, T.: Cryptographic hash-function basics: Definitions, implications, and separations for preimage resistance, second-preimage resistance, and collision resistance. In: International workshop on fast software encryption. pp. 371–388. Springer (2004)
24. Roy, S., Conti, M., Setia, S., Jajodia, S.: Secure data aggregation in wireless sensor networks: Filtering out the attacker’s impact. IEEE Transactions on Information Forensics and Security **9**(4), 681–694 (2014)
25. Shamir, A.: How to share a secret. Communications of the ACM **22**(11), 612–613 (1979)
26. Shi, J., Zhang, R., Liu, Y., Zhang, Y.: Prisense: privacy-preserving data aggregation in people-centric urban sensing systems. In: 2010 Proceedings IEEE INFOCOM. pp. 1–9. IEEE (2010)
27. Wang, Y., Yan, Z., Feng, W., Liu, S.: Privacy protection in mobile crowd sensing: a survey. World Wide Web **23**(1), 421–452 (2020)
28. Yao, A.C.C.: How to generate and exchange secrets. In: 27th annual symposium on foundations of computer science (Sfcs 1986). pp. 162–167. IEEE (1986)
29. Yao, Y., Liu, J., Xiong, N.N.: Privacy-preserving data aggregation in two-tiered wireless sensor networks with mobile nodes. Sensors **14**(11), 21174–21194 (2014)
30. Zhang, Y., Chen, Q., Zhong, S.: Privacy-preserving data aggregation in mobile phone sensing. IEEE Transactions on Information Forensics and Security **11**(5), 980–992 (2016)
31. Zhang, Z., Li, J., Yu, S., Makaya, C.: Safelearning: Secure aggregation in federated learning with backdoor detectability. IEEE Transactions on Information Forensics and Security **18**, 3289–3304 (2023)
32. Zhao, L., Jiang, J., Feng, B., Wang, Q., Shen, C., Li, Q.: Sear: Secure and efficient aggregation for byzantine-robust federated learning. IEEE Transactions on Dependable and Secure Computing **19**(5), 3329–3342 (2021)

## A Security Proofs

### A.1 Proof of Theorem 1

*Proof.* We construct a simulator  $\mathcal{S}$  and prove indistinguishability through two hybrids.

*Simulator.* Given  $L_{\mathcal{A},q}^{\text{coal}}$ , the simulator samples simulated setup material and runs  $\mathcal{A}$  with  $\text{inp}_{\mathcal{A},q}$  and  $\text{rand}_{\mathcal{A},q}$ . It constructs a simulated outcome vector  $\tilde{O}$  such that  $\text{type}(\tilde{O}) = \text{type}(O)$ . For every honest-sensor entry whose leaked type is  $\text{Hon}$ , it samples a fresh simulated submitted vector  $\tilde{r}_i$  from the pseudorandom label distribution and sets  $\tilde{o}_i = (\text{Hon}, \tilde{r}_i)$ . The same  $\tilde{r}_i$  is used consistently in the simulated submissions, local reports, evidence sets, first-layer checking

gates, and filter-gate inputs. For corrupted sensors and the corrupted server, all submissions, reports, and proposals are induced by running  $\mathcal{A}$ .

The PBFT-based agreement transcript is generated from the simulated submission layer: the simulator forms signed honest-server local reports, constructs or validates evidence sets  $\tilde{E}_i$ , and derives  $\tilde{o}_i = \text{Derive}(\tilde{E}_i)$  by the same deterministic evidence rule as the real protocol, and generates honest prepare/commit messages on  $H(\tilde{o}_i, \tilde{E}_i)$ . It then generates the first-layer checking behavior, signed status-vector messages, and client-side label-release transcripts consistently with  $u^*$ . The client releases to the corrupted server only the server-specific client-side filter-gate labels corresponding to the accepted majority status vector  $u^*$ .

The simulator invokes the garbled-circuit obliviousness simulator for the final garbled fusion circuit. It shares every simulated circuit-input label used in the simulated fusion-circuit evaluation with byte-wise (3,4) Shamir sharing, and constructs the server-specific filter gates, reconstruction transcript, second-layer checking gates, and final output-label messages so that reconstruction yields exactly these labels. For each complementary circuit-input label, any share value visible to the corrupted server is generated as an independent uniform field-element string of the same form, as justified by below-threshold byte-wise Shamir privacy.

If  $\text{abortinfo}_q$  specifies an abort, the simulator outputs only the visible transcript prefix up to that abort point.

#### *Hybrids.*

- Let  $H_0 = (\text{pp}, \text{View}_{\mathcal{A},q})$  be the real distribution.
- In  $H_1$ , replace the corrupted coalition's visible below-threshold shares of complementary circuit-input labels by uniformly random byte-wise shares. This does not change the labels used in evaluation, their reconstruction shares, checking outcomes, or the final fusion-circuit evaluator view. By byte-wise (3,4) Shamir privacy,  $H_0 \approx_s H_1$ .
- In  $H_2$ , replace the real honest-side interface by the simulated one described above. Honest sensor-side labels visible in submissions, evidence, checking gates, and filter-gate inputs are replaced by simulated pseudorandom labels, which is indistinguishable by PRG security. The PBFT transcript remains consistent because signatures are generated under simulated honest signing keys, corrupted-party messages are induced by  $\mathcal{A}$ , and prepare/commit digests bind the simulated outcomes and evidence by collision resistance of  $H$ . For the final fusion circuit,  $H_2$  replaces the real evaluator view  $(\hat{C}_{\text{fus}}, X_{\text{fus}}, Y_{\text{fus}})$  with a simulated one generated as follows:  $(\tilde{C}_{\text{fus}}, \tilde{X}_{\text{fus}}) \leftarrow \text{Sim}_{\text{GC}}^{\text{obv}}(1^\kappa, \Phi_{\text{fus}})$ ,  $\tilde{Y}_{\text{fus}} \leftarrow \text{Ev}(\tilde{C}_{\text{fus}}, \tilde{X}_{\text{fus}})$ . This replacement is computationally indistinguishable by the obliviousness of the garbling scheme for the final fusion circuit. The surrounding filter-gate outputs, reconstruction shares, and second-layer checking gates are regenerated coherently so that reconstruction yields exactly  $\tilde{X}_{\text{fus}}$ . Auxiliary checking and filter gates are generated so that every row evaluable by the corrupted coalition has the prescribed output. Garbled-gate authenticity

prevents useful outputs from invalid labels except with negligible probability. Therefore,  $H_1 \approx_c H_2$ .

By construction,  $H_2$  is exactly the output distribution of  $\mathcal{S}(1^\kappa, L_{\mathcal{A},q}^{\text{coal}})$ . Hence  $(\text{pp}, \text{View}_{\mathcal{A},q}) = H_0 \approx_s H_1 \approx_c H_2 = \mathcal{S}(1^\kappa, L_{\mathcal{A},q}^{\text{coal}})$ , which proves the theorem.